

Patrones de Arquitectura para Aplicaciones Web

1. Principios de arquitectura de software

La arquitectura de software define la organización fundamental de un sistema: sus componentes, las relaciones entre ellos, el entorno en el que operan y los principios que guían su diseño y evolución. Una buena arquitectura busca equilibrar los requisitos funcionales con los atributos de calidad (rendimiento, seguridad, mantenibilidad, escalabilidad).

Principios fundamentales

- Separación de responsabilidades (Separation of Concerns): cada componente debe encargarse de una única responsabilidad bien definida.
- Bajo acoplamiento y alta cohesión: los módulos deben depender lo mínimo posible entre sí, pero cada uno debe ser internamente coherente.
- Reutilización: diseñar componentes que puedan emplearse en distintos contextos sin duplicar lógica.
- Escalabilidad: capacidad del sistema de crecer (en usuarios, datos o carga) sin rediseñarse por completo.
- Mantenibilidad: facilidad para corregir errores, añadir funcionalidades o adaptar el sistema con el menor esfuerzo posible.
- Principio DRY (Don't Repeat Yourself) y principio KISS (Keep It Simple): evitar duplicación y complejidad innecesaria.

2. Arquitectura por capas

La arquitectura por capas (layered architecture) organiza el sistema en niveles horizontales, donde cada capa ofrece servicios a la capa superior y consume servicios de la capa inferior. Es uno de los patrones más utilizados en aplicaciones web tradicionales.

Capas típicas

- Capa de presentación: interfaz de usuario, responsable de mostrar la información y capturar la interacción del usuario.
- Capa de lógica de negocio (o de aplicación): contiene las reglas y procesos que definen el comportamiento del sistema.
- Capa de acceso a datos: gestiona la comunicación con la base de datos u otras fuentes de persistencia.
- Capa de datos: el propio almacenamiento (base de datos, sistema de archivos, etc.).

Ventajas y desventajas

- Ventaja: facilita el mantenimiento y permite sustituir una capa sin afectar a las demás, siempre que se respeten las interfaces.
- Ventaja: favorece la separación de responsabilidades y el trabajo en equipo por especialidades.
- Desventaja: puede introducir sobrecarga de rendimiento al atravesar varias capas para cada operación.
- Desventaja: un diseño rígido de capas puede dificultar cambios transversales que afectan a varias capas a la vez.

3. Arquitecturas en pizarra, dirigida por eventos y peer-to-peer

Arquitectura en pizarra (Blackboard)

Se basa en un espacio de datos compartido (la "pizarra") al que distintos componentes especializados acceden para leer y escribir información de forma colaborativa, sin comunicarse directamente entre sí. Es útil en problemas complejos sin una solución algorítmica determinista, como sistemas de inteligencia artificial o reconocimiento de patrones.

Arquitectura dirigida por eventos (Event-Driven)

Los componentes se comunican mediante la emisión y consumo de eventos, normalmente a través de un bus de eventos o intermediario de mensajería. Cuando ocurre un evento, los componentes suscritos reaccionan de forma asíncrona.

- Favorece el desacoplamiento entre productores y consumidores de eventos.
- Es especialmente adecuada para sistemas reactivos, de tiempo real o con procesamiento asíncrono (por ejemplo, notificaciones, sistemas de mensajería, IoT).
- Introduce complejidad adicional en la depuración y el seguimiento del flujo de ejecución.

Arquitectura peer-to-peer (P2P)

En este modelo no existe una distinción fija entre cliente y servidor: cada nodo (peer) puede actuar como cliente y como servidor simultáneamente, compartiendo recursos directamente con otros nodos de la red.

- Elimina el punto único de fallo asociado a un servidor central.
- Se emplea en sistemas de compartición de archivos, redes blockchain y aplicaciones de comunicación distribuida.
- Plantea retos de seguridad, control de versiones de datos y coordinación entre nodos.

4. Arquitectura Orientada a Servicios (SOA) y Microservicios

SOA (Service-Oriented Architecture)

SOA organiza el sistema como un conjunto de servicios independientes que se comunican entre sí, normalmente a través de protocolos estándar (SOAP, REST) y con frecuencia coordinados por un bus de servicios empresarial (ESB).

- Los servicios son reutilizables y suelen exponer contratos bien definidos (WSDL, por ejemplo).
- Favorece la integración entre sistemas heterogéneos de una organización.
- Suele implicar servicios de mayor tamaño (granularidad más gruesa) que los microservicios.

Microservicios

Es una evolución de SOA que propone dividir la aplicación en servicios pequeños, independientes y desplegables por separado, cada uno responsable de una funcionalidad de negocio concreta y con su propia base de datos cuando es posible.

- Cada microservicio puede desarrollarse, desplegarse y escalarse de forma independiente.
- La comunicación entre servicios suele hacerse mediante APIs REST, gRPC o mensajería asíncrona.
- Facilita el uso de distintas tecnologías por servicio (políglota), pero incrementa la complejidad operativa (despliegue, monitorización, gestión de fallos distribuidos).
- Requiere prácticas como contenedores (Docker), orquestación (Kubernetes) y patrones de resiliencia (circuit breaker, retry, timeout).

Comparativa SOA vs. Microservicios

Aspecto	SOA	Micro servicios
Granularidad	Servicios más grandes	Servicios pequeños y específicos
Comunicación	ESB, SOAP/REST	REST, gRPC, mensajería
Base de datos	Frecuentemente compartida	Independiente por servicio
Despliegue	Conjunto, más acoplado	Independiente por servicio
Complejidad operativa	Moderada	Alta (requiere orquestación)